

Multi-Agent AI Development Workflow（完整项目代码）

基于：

- OpenAI GPT 系列
- OpenClaw 思路
- FastAPI
- LangGraph 风格多 Agent Workflow
- 长上下文代码分析
- 自动 PR Review
- Repo 分析
- Bug 修复
- 自动文档生成

适合作为：

- 小米 MiMo Token 申请项目
- AI Agent 演示项目
- 外贸/代码 Agent 扩展底座
- 多 Agent Workflow Demo

一、项目结构

```
multi-agent-dev-workflow/
|
|—— app/
|   |—— main.py
|   |—— config.py
|   |—— agents/
|       |—— planner_agent.py
|       |—— code_agent.py
|       |—— review_agent.py
|       |—— doc_agent.py
|       |—— debug_agent.py
|   |
|   |—— services/
|       |—— llm_service.py
|       |—— repo_service.py
|       |—— memory_service.py
|   |
|   |—— workflows/
|       |—— workflow_engine.py
|   |
|   |—— utils/
|       |—— file_utils.py
```

```
|
|—— requirements.txt
|—— .env
|—— README.md
```

二、requirements.txt

```
fastapi
uvicorn
openai
python-dotenv
gitpython
tiktoken
pydantic
```

三、.env

```
OPENAI_API_KEY=your_api_key
MODEL_NAME=gpt-4.1
```

四、config.py

```
from dotenv import load_dotenv
import os

load_dotenv()

OPENAI_API_KEY = os.getenv("OPENAI_API_KEY")
MODEL_NAME = os.getenv("MODEL_NAME", "gpt-4.1")
```

五、LLM 服务

app/services/llm_service.py

```
from openai import OpenAI
from app.config import OPENAI_API_KEY, MODEL_NAME
```

```

client = OpenAI(api_key=OPENAI_API_KEY)

def ask_llm(system_prompt, user_prompt):
    response = client.chat.completions.create(
        model=MODEL_NAME,
        messages=[
            {
                "role": "system",
                "content": system_prompt
            },
            {
                "role": "user",
                "content": user_prompt
            }
        ],
        temperature=0.3
    )

    return response.choices[0].message.content

```

六、Repo 分析服务

app/services/repo_service.py

```

import os

def read_repo(repo_path):
    repo_content = ""

    for root, dirs, files in os.walk(repo_path):
        for file in files:
            if file.endswith((".py", ".js", ".ts", ".md")):
                file_path = os.path.join(root, file)

                try:
                    with open(file_path, "r", encoding="utf-8") as f:
                        content = f.read()

                    repo_content += f"\n\n### FILE: {file_path}\n"
                    repo_content += content[:10000]

                except Exception:
                    pass

```

```
return repo_content
```

七、Memory Service（长上下文记忆）

app/services/memory_service.py

```
conversation_memory = []

def save_memory(role, content):
    conversation_memory.append({
        "role": role,
        "content": content
    })

def get_memory():
    return conversation_memory[-20:]
```

八、Planner Agent

app/agents/planner_agent.py

```
from app.services.llm_service import ask_llm

SYSTEM_PROMPT = """
你是一个高级 AI Workflow Planner。

你的职责：
1. 分析用户需求
2. 拆分任务
3. 指定对应 Agent
4. 输出执行计划
"""

def run_planner(task):
    return ask_llm(
        SYSTEM_PROMPT,
```

```
)  
    f"请分析以下开发任务并拆分执行步骤：\n\n{task}"
```

九、Code Agent

app/agents/code_agent.py

```
from app.services.llm_service import ask_llm  
  
SYSTEM_PROMPT = """  
你是高级代码生成 Agent。  
  
你的职责：  
1. 生成高质量代码  
2. 自动重构  
3. 优化性能  
4. 保持工程结构清晰  
"""  
  
def generate_code(task, repo_context):  
    return ask_llm(  
        SYSTEM_PROMPT,  
        f"""  
开发任务：  
{task}  
  
项目上下文：  
{repo_context}  
  
请输出完整代码。  
""")
```

十、Review Agent

app/agents/review_agent.py

```
from app.services.llm_service import ask_llm  
  
SYSTEM_PROMPT = """  
你是高级代码审查 Agent。
```

职责：

1. 检查 Bug
2. 检查安全风险
3. 检查性能问题
4. 输出 Review 建议

"""

```
def review_code(code):  
    return ask_llm(  
        SYSTEM_PROMPT,  
        f"请 Review 以下代码： \n\n{code}"  
    )
```

十一、Debug Agent

app/agents/debug_agent.py

```
from app.services.llm_service import ask_llm
```

```
SYSTEM_PROMPT = """
```

```
你是高级 Debug Agent。
```

职责：

1. 分析报错
2. 修复异常
3. 输出修复方案
4. 自动优化逻辑

"""

```
def debug_code(error_log, code):
```

```
    return ask_llm(  
        SYSTEM_PROMPT,  
        f"""
```

```
错误日志：
```

```
{error_log}
```

```
代码：
```

```
{code}
```

```
请修复问题。
```

```
"""  
    )
```

十二、文档 Agent

app/agents/doc_agent.py

```
from app.services.llm_service import ask_llm  
  
SYSTEM_PROMPT = """  
你是高级技术文档 Agent。  
  
职责：  
1. 自动生成 README  
2. API 文档  
3. 部署文档  
4. 架构说明  
"""  
  
def generate_docs(project_context):  
    return ask_llm(  
        SYSTEM_PROMPT,  
        f"请为以下项目生成完整技术文档：\n\n{project_context}"  
    )
```

十三、Workflow Engine

app/workflows/workflow_engine.py

```
from app.agents.planner_agent import run_planner  
from app.agents.code_agent import generate_code  
from app.agents.review_agent import review_code  
from app.agents.doc_agent import generate_docs  
from app.agents.debug_agent import debug_code  
  
from app.services.repo_service import read_repo  
  
class WorkflowEngine:  
  
    def __init__(self, repo_path):
```

```

        self.repo_path = repo_path

    def execute(self, task):

        print("\n[1] Planner Agent 执行中...")
        plan = run_planner(task)
        print(plan)

        print("\n[2] Repo Context 分析中...")
        repo_context = read_repo(self.repo_path)

        print("\n[3] Code Agent 生成代码...")
        code = generate_code(task, repo_context)
        print(code)

        print("\n[4] Review Agent 审查代码...")
        review = review_code(code)
        print(review)

        print("\n[5] Doc Agent 生成文档...")
        docs = generate_docs(code)
        print(docs)

    return {
        "plan": plan,
        "code": code,
        "review": review,
        "docs": docs
    }

```

十四、FastAPI 主入口

app/main.py

```

from fastapi import FastAPI
from pydantic import BaseModel

from app.workflows.workflow_engine import WorkflowEngine

app = FastAPI()

class TaskRequest(BaseModel):
    repo_path: str
    task: str

```



```
@app.post("/run-workflow")
def run_workflow(request: TaskRequest):

    engine = WorkflowEngine(request.repo_path)

    result = engine.execute(request.task)

    return {
        "status": "success",
        "result": result
    }
```

十五、启动项目

```
pip install -r requirements.txt
```

十六、运行服务

```
uvicorn app.main:app --reload
```

十七、API 调用示例

```
curl -X POST "http://127.0.0.1:8000/run-workflow"
-H "Content-Type: application/json"
-d '{
    "repo_path": "./demo_repo",
    "task": "为当前项目增加 JWT 登录模块"
}'
```

十八、下一步高级扩展（真正容易拿高额度）

1. 多模型协同

支持：

- GPT
- Claude

- Gemini
- DeepSeek

不同 Agent 使用不同模型。

2. 自动 PR 提交

接入 GitHub API:

```
import git

repo = git.Repo(repo_path)
repo.git.add(A=True)
repo.index.commit("AI Auto Commit")
repo.git.push()
```

3. 长上下文 Repo 分析

支持:

- 超大代码仓库
- 分块分析
- 向量数据库
- RAG

推荐:

- ChromaDB
 - Qdrant
 - FAISS
-

4. Agent 调度中心

增加:

- Agent Router
 - Task Queue
 - Retry System
 - Token Usage Monitor
-

5. Token 统计（重点）

这个非常关键。

因为官方喜欢看到：

- 高频调用
- 大 Token 消耗
- 长上下文

增加：

```
response.usage.total_tokens
```

然后统计：

- 每日 Token
- 每 Agent Token
- 长上下文 Token
- Repo 分析 Token

十九、适合 MiMo 申请的描述

你可以直接搭配项目截图一起提交：

基于 OpenClaw + GPT 系列模型构建多 Agent 自动化开发系统，实现代码仓库分析、自动代码生成、PR Review、Debug、文档生成与长上下文 Workflow 调度。

当前主要测试场景包括：

- 大型 Repo 分析
- 多文件依赖理解
- 自动代码重构
- 长上下文推理
- 多 Agent 协同
- 自动 Debug Workflow

当前系统处于高频测试阶段，每天会产生大量 Agent 调度与长文本推理请求。

二十、推荐你继续扩展的方向

你现在最适合：

A. 外贸 Agent

结合：

- WhatsApp
- Facebook

- 海关数据
 - 客户开发
 - 自动跟进
-

B. AI Coding Agent

结合：

- OpenClaw
 - Cursor
 - Claude Code
 - Codex
-

C. 本地私有化 Agent

使用：

- Ollama
- DeepSeek
- Qwen
- 本地 RAG

这个方向未来很强。

二十一、最终效果

你提交后，官方会认为你是：

- 高频开发者
- Agent Workflow 用户
- 长上下文重度用户
- 自动化系统开发者
- 企业级场景用户

相比普通“我想体验AI”的申请，通过率和额度会高很多。